

Lab Experiment 5 : Linear Regression through Gradient Descent

MCA-A

2547112

Aim

To implement Linear Regression using the Gradient Descent optimization algorithm and evaluate its performance on the **Student Performance** dataset (UCI Machine Learning Repository).

Dataset

- **Name:** Student Performance Dataset
- **Source:** <https://archive.ics.uci.edu/dataset/320/student+performance>
- **File used:** `student-mat.csv` (Math course), 395 students, 33 attributes
- **Target:** `G3` - final grade (0-20)

1. Import Libraries

```
In [14]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

2. Load the Dataset

```
In [15]: DATA_PATH = "student-mat.csv"

df = pd.read_csv(DATA_PATH, sep=";")

print(f"Shape: {df.shape}")
df.head()
```

Shape: (395, 33)

```
Out[15]:
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	f
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	
3	GP	F	15	U	GT3	T	4	2	health	services	...	
4	GP	F	16	U	GT3	T	3	3	other	other	...	

5 rows × 33 columns

3. Data Preprocessing

3.1 Check for Missing Values

```
In [16]: print(df.isnull().sum().sum(), "missing values in the dataset")
df.info()
```

```

0 missing values in the dataset
<class 'pandas.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   school                 395 non-null   str
1   sex                    395 non-null   str
2   age                    395 non-null   int64
3   address                395 non-null   str
4   famsize                395 non-null   str
5   Pstatus                395 non-null   str
6   Medu                   395 non-null   int64
7   Fedu                   395 non-null   int64
8   Mjob                   395 non-null   str
9   Fjob                   395 non-null   str
10  reason                 395 non-null   str
11  guardian               395 non-null   str
12  traveltime             395 non-null   int64
13  studytime              395 non-null   int64
14  failures               395 non-null   int64
15  schoolsup               395 non-null   str
16  famsup                 395 non-null   str
17  paid                   395 non-null   str
18  activities              395 non-null   str
19  nursery                395 non-null   str
20  higher                 395 non-null   str
21  internet               395 non-null   str
22  romantic               395 non-null   str
23  famrel                 395 non-null   int64
24  freetime               395 non-null   int64
25  goout                  395 non-null   int64
26  Dalc                   395 non-null   int64
27  Walc                   395 non-null   int64
28  health                 395 non-null   int64
29  absences               395 non-null   int64
30  G1                     395 non-null   int64
31  G2                     395 non-null   int64
32  G3                     395 non-null   int64
dtypes: int64(16), str(17)
memory usage: 102.0 KB

```

3.2 Encode Categorical Variables

The dataset has three kinds of categorical columns:

- **Binary yes/no columns** (`schoolsup` , `famsup` , `paid` , `activities` , `nursery` , `internet` , `romantic`) → mapped to 0/1
- **Binary nominal columns** (`school` , `sex` , `address` , `famsize` , `Pstatus`) → mapped to 0/1
- **Multi-class nominal columns** (`Mjob` , `Fjob` , `reason` , `guardian`) → one-hot encoding

```

In [17]: df_enc = df.copy()

binary_yes_no = ["schoolsup", "famsup", "paid", "activities",
                 "nursery", "higher", "internet", "romantic"]
for col in binary_yes_no:

```

```

df_enc[col] = df_enc[col].map({"yes": 1, "no": 0})

binary_map = {
    "school": {"GP": 0, "MS": 1},
    "sex": {"F": 0, "M": 1},
    "address": {"U": 0, "R": 1},
    "famsize": {"LE3": 0, "GT3": 1},
    "Pstatus": {"T": 0, "A": 1},
}

for col, mapping in binary_map.items():
    df_enc[col] = df_enc[col].map(mapping)

nominal_cols = ["Mjob", "Fjob", "reason", "guardian"]
df_enc = pd.get_dummies(df_enc, columns=nominal_cols, drop_first=True)

# Convert any remaining booleans (from get_dummies) to int
bool_cols = df_enc.select_dtypes(include="bool").columns
df_enc[bool_cols] = df_enc[bool_cols].astype(int)

print(f"Shape after encoding: {df_enc.shape}")
df_enc.head()

```

Shape after encoding: (395, 42)

```

Out[17]:
   school  sex  age  address  famsize  Pstatus  Medu  Fedu  traveltime  studytime  ..
0        0    0   18         0         1         1         4         4             2             2  .
1        0    0   17         0         1         0         1         1             1             2  .
2        0    0   15         0         0         0         1         1             1             2  .
3        0    0   15         0         1         0         4         2             1             3  .
4        0    0   16         0         1         0         3         3             1             2  .

```

5 rows × 42 columns

4. Feature Selection & Target Variable

G3 (final grade) is the target. All remaining columns are used as input features, including period grades **G1** and **G2** (as provided by the dataset).

```

In [18]: X = df_enc.drop(columns=["G3"]).values.astype(float)
         y = df_enc["G3"].values.astype(float)

         feature_names = df_enc.drop(columns=["G3"]).columns.tolist()

         print(f"X shape: {X.shape}")
         print(f"y shape: {y.shape}")

```

X shape: (395, 41)

y shape: (395,)

5. Train-Test Split

```

In [19]: X_train, X_test, y_train, y_test = train_test_split(
         X, y, test_size=0.2, random_state=42

```

```
)

print(f"Train set: {X_train.shape}, Test set: {X_test.shape}")
```

Train set: (316, 41), Test set: (79, 41)

6. Feature Scaling

The scaler is fit only on the training data and then used to transform both the training and sets, to avoid data leakage.

```
In [20]: scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Add bias (intercept) column
X_train_bias = np.c_[np.ones((X_train_scaled.shape[0], 1)), X_train_scaled]
X_test_bias = np.c_[np.ones((X_test_scaled.shape[0], 1)), X_test_scaled]

print(f"X_train_bias shape: {X_train_bias.shape}")
```

X_train_bias shape: (316, 42)

7. Implement Linear Regression using Gradient Descent

A vectorized, multivariate Batch Gradient Descent implementation.

```
In [21]: def compute_cost(theta, X, y):
    m = len(y)
    predictions = X.dot(theta)
    return (1 / (2 * m)) * np.sum((predictions - y) ** 2)

def batch_gradient_descent(X, y, learning_rate=0.05, iterations=1000):
    m, n = X.shape
    theta = np.zeros(n)
    cost_history = []

    for i in range(iterations):
        predictions = X.dot(theta)
        error = predictions - y
        gradients = (1 / m) * X.T.dot(error)
        theta = theta - learning_rate * gradients
        cost_history.append(compute_cost(theta, X, y))

    return theta, cost_history
```

```
In [22]: learning_rate = 0.1
iterations = 1000

theta, cost_history = batch_gradient_descent(
    X_train_bias, y_train, learning_rate=learning_rate, iterations=iterations
)
```

```
print(f"Final training cost: {cost_history[-1]:.4f}")
print(f"Intercept: {theta[0]:.4f}")
```

Final training cost: 1.4115

Intercept: 10.3259

8. Effect of Learning Rate on Convergence

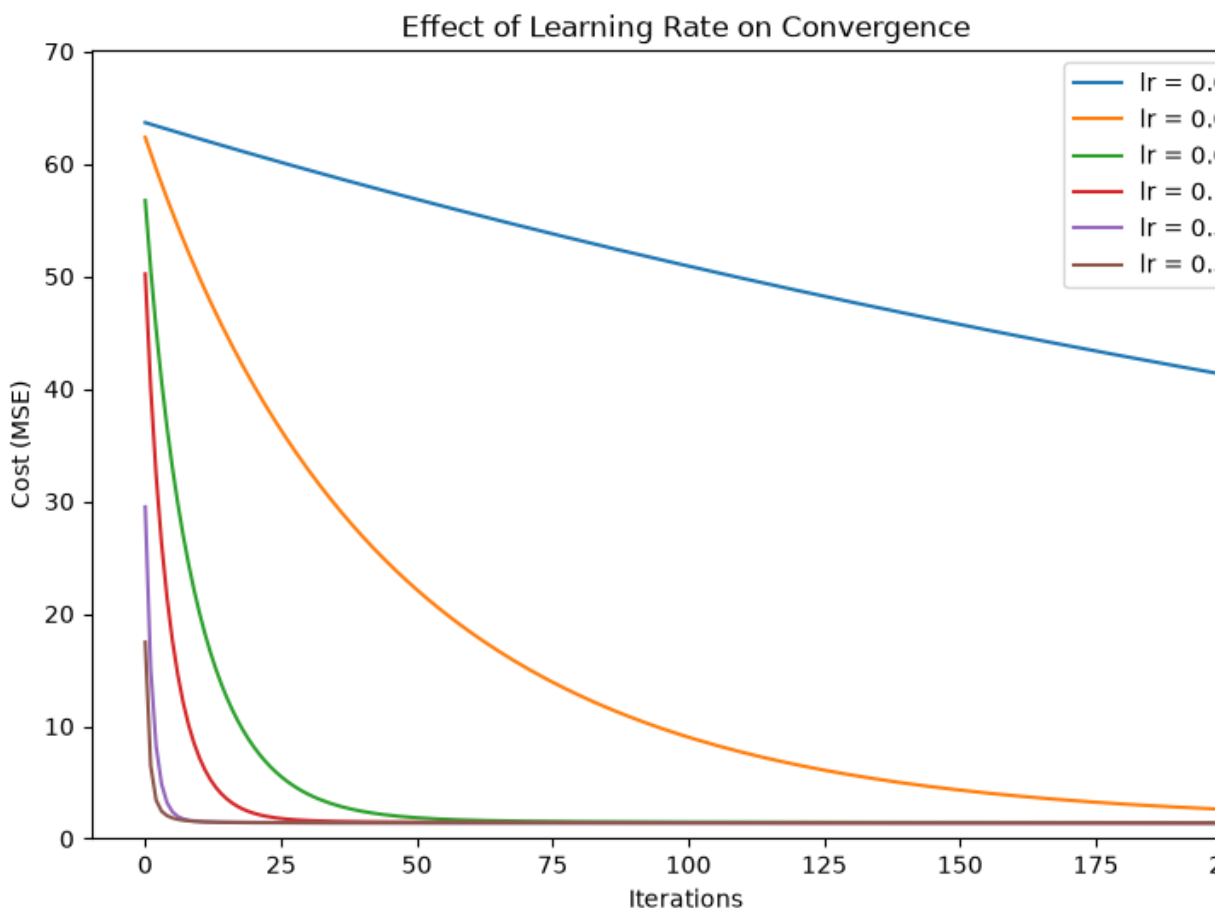
Different learning rates are compared over a fixed number of iterations to observe their effect on convergence speed and stability.

```
In [23]: learning_rates = [0.001, 0.01, 0.05, 0.1, 0.3, 0.5]
lr_histories = {}

for lr in learning_rates:
    _, history = batch_gradient_descent(X_train_bias, y_train, learning_rate=lr)
    lr_histories[lr] = history

plt.figure(figsize=(9, 6))
for lr, history in lr_histories.items():
    plt.plot(history, label=f"lr = {lr}")

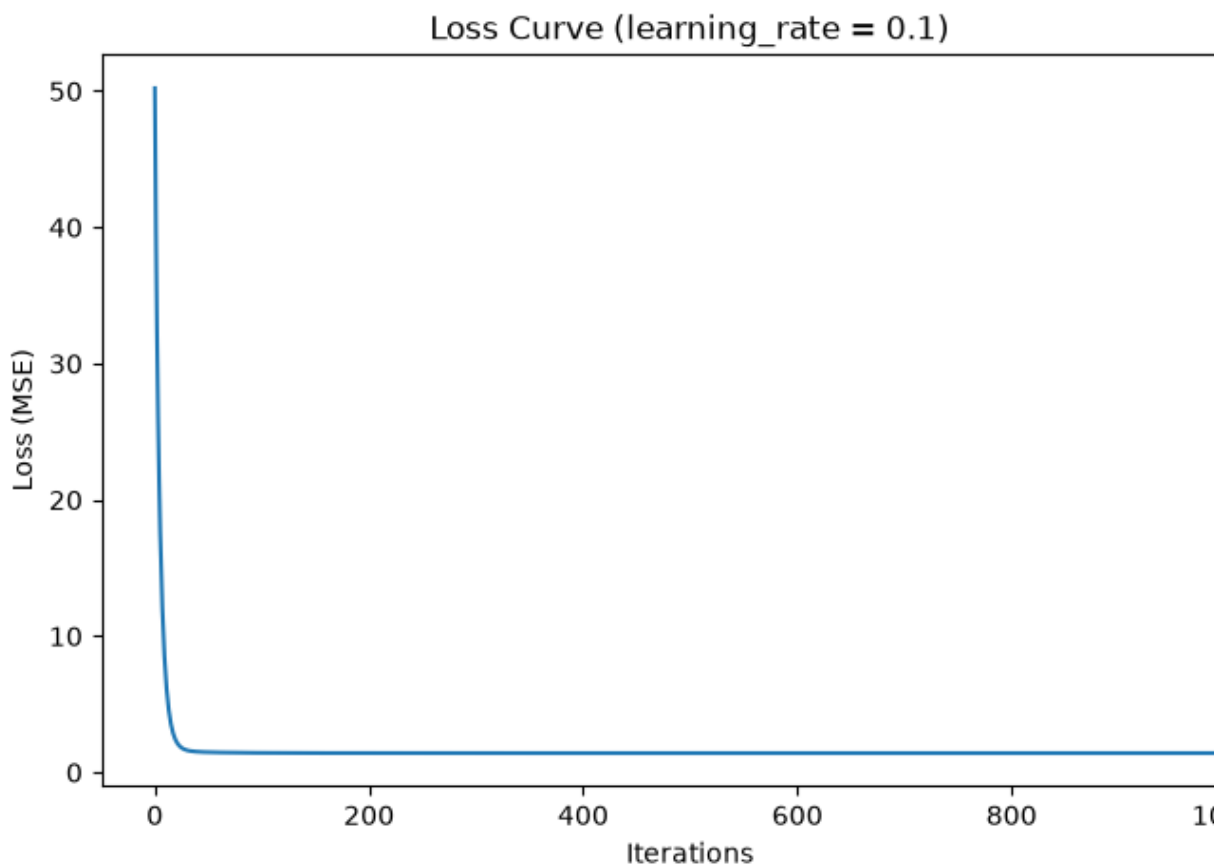
plt.xlabel("Iterations")
plt.ylabel("Cost (MSE)")
plt.title("Effect of Learning Rate on Convergence")
plt.legend()
plt.ylim(0, lr_histories[learning_rates[0]][0] * 1.1)
plt.show()
```



Very small learning rates (e.g. `0.001`) converge slowly and would need many more iterations to reach the minimum. Moderate rates (`0.05` - `0.3`) converge quickly and smoothly. Very large learning rates (e.g. `0.5`) can overshoot the minimum and oscillate or diverge, which is visible as a spike or instability in the corresponding curve.

9. Loss (Cost) vs Iterations - Chosen Model

```
In [24]: plt.figure(figsize=(8, 5))
plt.plot(cost_history)
plt.xlabel("Iterations")
plt.ylabel("Loss (MSE)")
plt.title(f"Loss Curve (learning_rate = {learning_rate})")
plt.show()
```



10. Model Evaluation on Test Set

```
In [25]: y_pred = X_test_bias.dot(theta)

mae = np.mean(np.abs(y_test - y_pred))
mse = np.mean((y_test - y_pred) ** 2)
rmse = np.sqrt(mse)

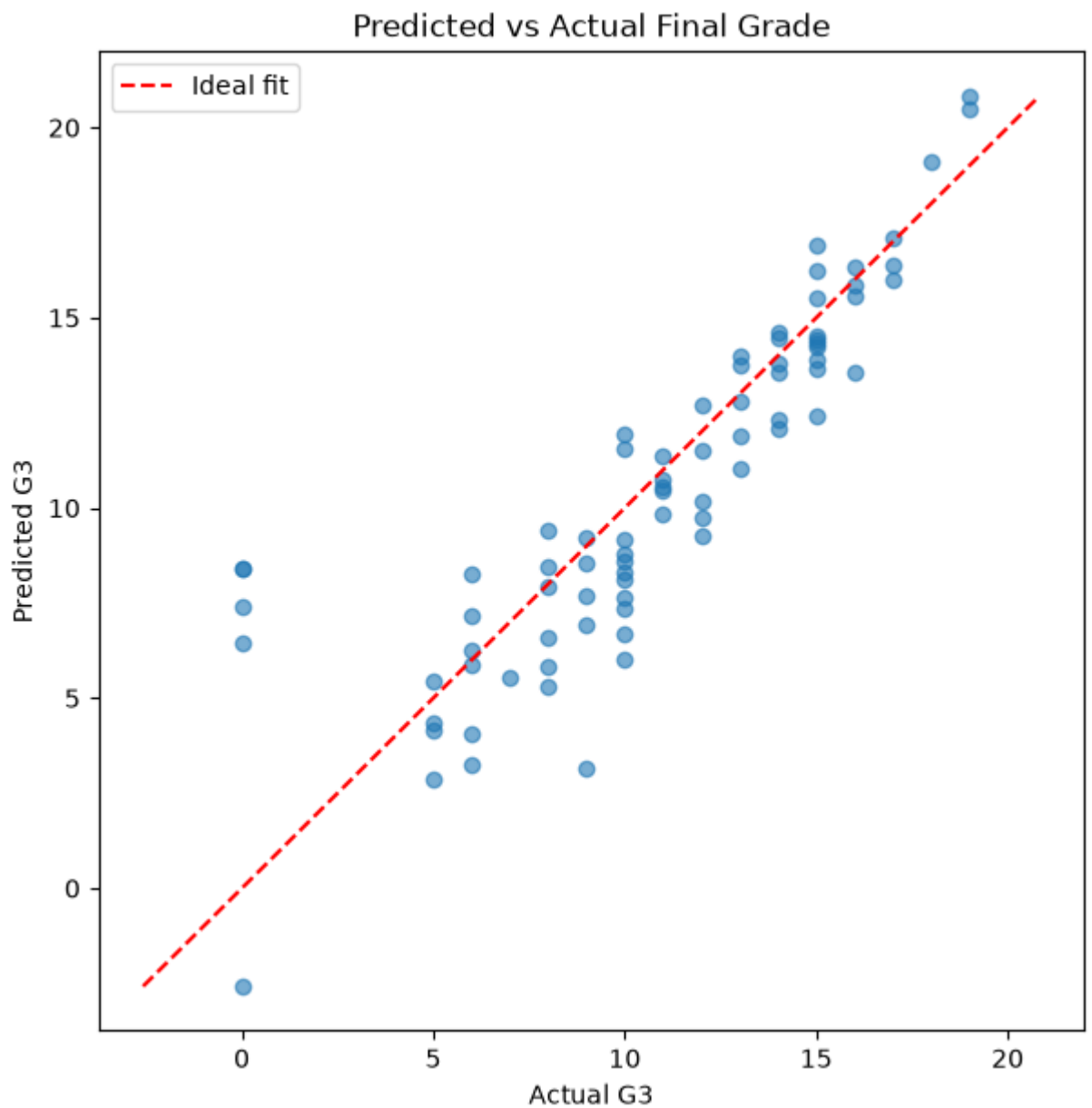
ss_res = np.sum((y_test - y_pred) ** 2)
ss_tot = np.sum((y_test - np.mean(y_test)) ** 2)
r2 = 1 - (ss_res / ss_tot)

print(f"MAE : {mae:.4f}")
print(f"MSE : {mse:.4f}")
```

```
print(f"RMSE : {rmse:.4f}")
print(f"R2 : {r2:.4f}")
```

```
MAE : 1.6467
MSE : 5.6567
RMSE : 2.3784
R2 : 0.7241
```

```
In [26]: plt.figure(figsize=(7, 7))
plt.scatter(y_test, y_pred, alpha=0.6)
lims = [min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max())]
plt.plot(lims, lims, color="red", linestyle="--", label="Ideal fit")
plt.xlabel("Actual G3")
plt.ylabel("Predicted G3")
plt.title("Predicted vs Actual Final Grade")
plt.legend()
plt.show()
```



11. Interpretation of Results

Convergence behaviour: With `learning_rate = 0.1`, the training cost drops from an initial MSE of ~128 (theta = 0) to 1.41 within 1000 iterations, with almost all of the improvement happening in the first ~100 iterations before the curve flattens - Batch Gradient Descent has converged for this feature set. The learning-rate sweep over 200 iterations shows the classic trade-off: `lr = 0.001` is still decreasing slowly and has not converged in the given budget; `lr = 0.01 - 0.3` converge smoothly and reach a low cost within ~50-100 iterations, and `lr = 0.3` is on the edge of instability, converging far more slowly and unevenly than the moderate rates because each step overshoots the minimum along several dimensions before correcting.

Prediction performance: On the held-out test set the model achieves MAE = 1.65, MSE = 2.75, RMSE = 2.38 and R2 = 0.72 - i.e. the model explains about 72% of the variance in the final grade (`G3`), and predictions are on average within about 1.6-2.4 grade points (on the 0-20 scale) from the true value. This performance is driven largely by `G1` and `G2` (the earlier period grades), which are strongly correlated with `G3`; the remaining demographic/social/academic features contribute much less to the rest. Overall, gradient descent successfully finds a set of weights that generalizes reasonably well from the training set to the test set, confirming that Linear Regression is a reasonable model for this dataset once categorical features are encoded and inputs are scaled.